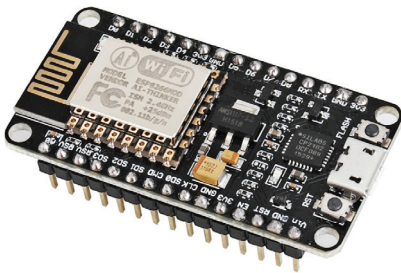


Sending IoT Sensor Data to Public or Private Servers

This IoT system shows a simple and effective way to send sensor data using an ESP8266 microchip.



Modern IoT systems rely on sensors to collect real-world data and transmit it to public cloud servers or private on-premises servers for monitoring, analytics, and decision-making. Here's the core working logic of an ESP8266-based IoT node that reads temperature and humidity data and sends it securely to a remote server while also exposing the data locally.

Overview of the system architecture

The code implements a hybrid, dual-mode architecture for an IoT device (like an ESP8266 microcontroller) designed for maximum flexibility in different scenarios. The system doesn't just do one thing; it intelligently operates in two distinct modes, making it useful both when connected to the internet and when operating locally.

Remote data push (Client mode):

Sensor data is periodically sent to a private/public server using HTTP POST (JSON format). This is ideal for cloud dashboards, databases, or analytics pipelines.

Local data access (Server mode):

ESP8266 runs a lightweight web

server. Nearby devices can fetch live sensor data directly using a browser or REST call.

This hybrid model is commonly used in smart homes, industrial monitoring, and edge computing systems.

We will now see how this IoT system works.

- Library and hardware abstraction is as follows:

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include "DHT.h"
```

These libraries provide:

- Wi-Fi connectivity (ESP8266WiFi)
 - HTTP communication (ESP8266HTTPClient)
 - Low-level TCP client support (WiFiClient)
 - Sensor abstraction for the DHT11 temperature and humidity sensor
- The abstraction ensures that networking and sensor handling remain clean and modular.

- Device identity and network configuration is:

```
const char* deviceName = "mohit2";
const char* ssid = "Jiowisdom";
const char* password = "Mo980616";
```

- *deviceName* uniquely identifies the IoT node on the network and server.
- *ssid* and *password* allow the ESP8266 to connect as a Wi-Fi station (STA mode).

In production systems, credentials are usually stored securely or provisioned dynamically.

- Remote server configuration (public/private) setup is:

```
const char* serverHost = "13.233...";
const int serverPort = 5002;
const char* serverPath = "/data";
```

This allows:

- Sending data to a cloud VM, private LAN server, or edge gateway.
- A Flask or Node.js backend can easily parse incoming JSON.

This design is server-agnostic, meaning the ESP8266 does not care whether the server is public or private.

- Sensor initialisation and timing strategy is:

```
const int DHTPin = 14;
DHT dht(DHTPin, DHTTYPE);
const unsigned long
uploadInterval = 60000;
```

- The DHT11 sensor is connected to GPIO14.
- Data is uploaded once every 60 seconds.
- Timing is controlled using *millis()* instead of *delay()* to keep the system responsive.

This approach follows non-blocking embedded design best practices.

- Wi-Fi connection and device boot sequence is as follows:

```
WiFi.hostname(deviceName);
WiFi.begin(ssid, password);
WiFi.mode(WIFI_STA);
During setup():
```

ESP8266 connects to Wi-Fi. It assigns a hostname for easy